

Explain What is Docker? How to use it commands
Interview Questions asked for 2yrs JAVA Developer

Docker Explained for Java Developers 🐳

What is Docker?

Docker is a **containerization platform** that packages your application and all its dependencies (libraries, configs, runtime) into a lightweight, portable unit called a **container**.

Think of it like this:

- 🏠 Traditional deployment = Moving your entire house
- 📦 Docker = Packing only what you need into a shipping container

Why Docker for Java?

- Eliminates "**works on my machine**" problems
- Run the same JAR/WAR across dev, staging, and production
- No need to install JDK on every server — it's in the container
- Microservices-friendly (Spring Boot + Docker = ❤️)

Core Concepts

Concept	Description	Java Analogy
Image	Blueprint/template	Like a <code>.class</code> file
Container	Running instance of an image	Like a running JVM process
Dockerfile	Instructions to build an image	Like a build script
Docker Hub	Registry to store/share images	Like Maven Central
Volume	Persistent storage for containers	Like a database file outside the app
Network	Communication between containers	Like microservice endpoints

Essential Docker Commands

🔧 Image Commands

bash

Pull an image from Docker Hub

```
docker pull openjdk:17
```

List all local images

```
docker images
```

Build image from Dockerfile

```
docker build -t my-java-app:1.0 .
```

Remove an image

```
docker rmi my-java-app:1.0
```

Tag an image

```
docker tag my-java-app:1.0 myrepo/my-java-app:latest
```

Container Commands

bash

Run a container

```
docker run -d -p 8080:8080 --name my-app my-java-app:1.0
```

List running containers

```
docker ps
```

List all containers (including stopped)

```
docker ps -a
```

Stop a container

```
docker stop my-app
```

Start a stopped container

```
docker start my-app
```

Restart a container

```
docker restart my-app
```

Remove a container

```
docker rm my-app
```

Force remove running container

```
docker rm -f my-app
```

Logs & Debugging

bash

```
# View container logs
docker logs my-app

# Follow live logs
docker logs -f my-app

# Execute command inside running container
docker exec -it my-app bash

# Inspect container details (IP, env vars, etc.)
docker inspect my-app

# Container resource usage (CPU, Memory)
docker stats my-app
```

Volume Commands

```
bash

# Create a volume
docker volume create my-data

# List volumes
docker volume ls

# Mount volume while running
docker run -v my-data:/app/data my-java-app:1.0

# Remove volume
docker volume rm my-data
```

Network Commands

```
bash

# List networks
docker network ls

# Create a custom network
docker network create my-network

# Run container on specific network
docker run --network my-network my-java-app:1.0

# Connect container to network
docker network connect my-network my-app
```

dockerfile

```
# Stage 1: Build
FROM maven:3.9-openjdk-17 AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run
FROM openjdk:17-jdk-slim
WORKDIR /app

# Copy JAR from builder stage
COPY --from=builder /app/target/*.jar app.jar

# Expose port
EXPOSE 8080

# Run the JAR
ENTRYPOINT ["java", "-jar", "app.jar"]
```

bash

```
# Build and run
docker build -t spring-app:1.0 .
docker run -d -p 8080:8080 --name spring-app spring-app:1.0
```

Docker Compose (Multi-container)

yaml

```
# docker-compose.yml
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - SPRING_DATASOURCE_URL=jdbc:mysql://db:3306/mydb
    depends_on:
      - db

  db:
    image: mysql:8.0
```

```
environment:
  MYSQL_ROOT_PASSWORD: root
  MYSQL_DATABASE: mydb
volumes:
  - db-data:/var/lib/mysql
```

```
volumes:
  db-data:
```

```
bash

# Start all services
docker-compose up -d

# Stop all services
docker-compose down

# View logs
docker-compose logs -f
```

Top Interview Questions for 2 YOE Java Developer

Q1. What is the difference between a Docker Image and a Container?

Image = static blueprint (like a class). **Container** = running instance (like an object).
You can create multiple containers from one image.

Q2. What is the difference between `CMD` and `ENTRYPOINT` in Dockerfile?

ENTRYPOINT defines the fixed command that always runs (e.g., `java -jar`).
CMD provides default arguments that can be overridden at runtime. In Spring Boot, use `ENTRYPOINT`.

Q3. What is Multi-Stage Build and why use it?

It uses multiple `FROM` statements — one stage to compile/build (with Maven/JDK), another to run (with slim JRE). Result: **smaller final image** without build tools bloating production.

Q4. How does Docker differ from a Virtual Machine?

Docker	VM
Shares host OS kernel	Has its own OS
Lightweight (MBs)	Heavy (GBs)
Starts in seconds	Starts in minutes

Q5. What is Docker Compose and when would you use it?

Docker Compose manages **multi-container apps** via a YAML file. For Java microservices, you'd use it to spin up your Spring Boot app + MySQL + Redis together with one command.

Q6. How do you pass environment variables to a Docker container?

```
bash

# Inline
docker run -e SPRING_PROFILES_ACTIVE=prod my-app

# From .env file
docker run --env-file .env my-app
```

Q7. What is a Docker Volume and why is it important?

Volumes provide **persistent storage** outside the container lifecycle. Without volumes, data (like DB files) is lost when a container is removed.

Q8. Explain Docker Networking modes.

Mode	Use Case
<code>bridge</code> (default)	Container-to-container on same host
<code>host</code>	Container shares host network directly
<code>none</code>	No networking
<code>overlay</code>	Multi-host (Docker Swarm/K8s)

Q9. How do you optimize a Docker image for a Java application?

- Use **multi-stage builds**
- Use slim base images (`openjdk:17-jdk-slim`)
- Add `.dockerignore` to exclude unnecessary files
- Layer caching — copy `pom.xml` before `src/` so dependencies are cached

Q10. What happens when you run `docker run -p 8080:8080`?

It maps **host port 8080** → **container port 8080**. Format is `HOST:CONTAINER`. So `localhost:8080` on your machine routes into the container.

Quick Cheat Sheet

```
bash

docker build -t app .           # Build image
docker run -d -p 8080:8080 app  # Run container
docker ps                       # List running
docker logs -f app              # Follow logs
docker exec -it app bash        # Shell into container
docker stop app                 # Stop
docker rm app                   # Remove
docker-compose up -d            # Start all services
docker-compose down             # Stop all services
```

This covers everything a 2-year Java developer would need for a Docker interview! Focus especially on **multi-stage builds**, **Docker Compose**, and **volumes** as these are commonly asked in practical rounds.

Explain What is Kubernetes? How to use it commands Interview Questions asked for 2yrs JAVA Developer

Kubernetes (K8s) for Java Developers

What is Kubernetes?

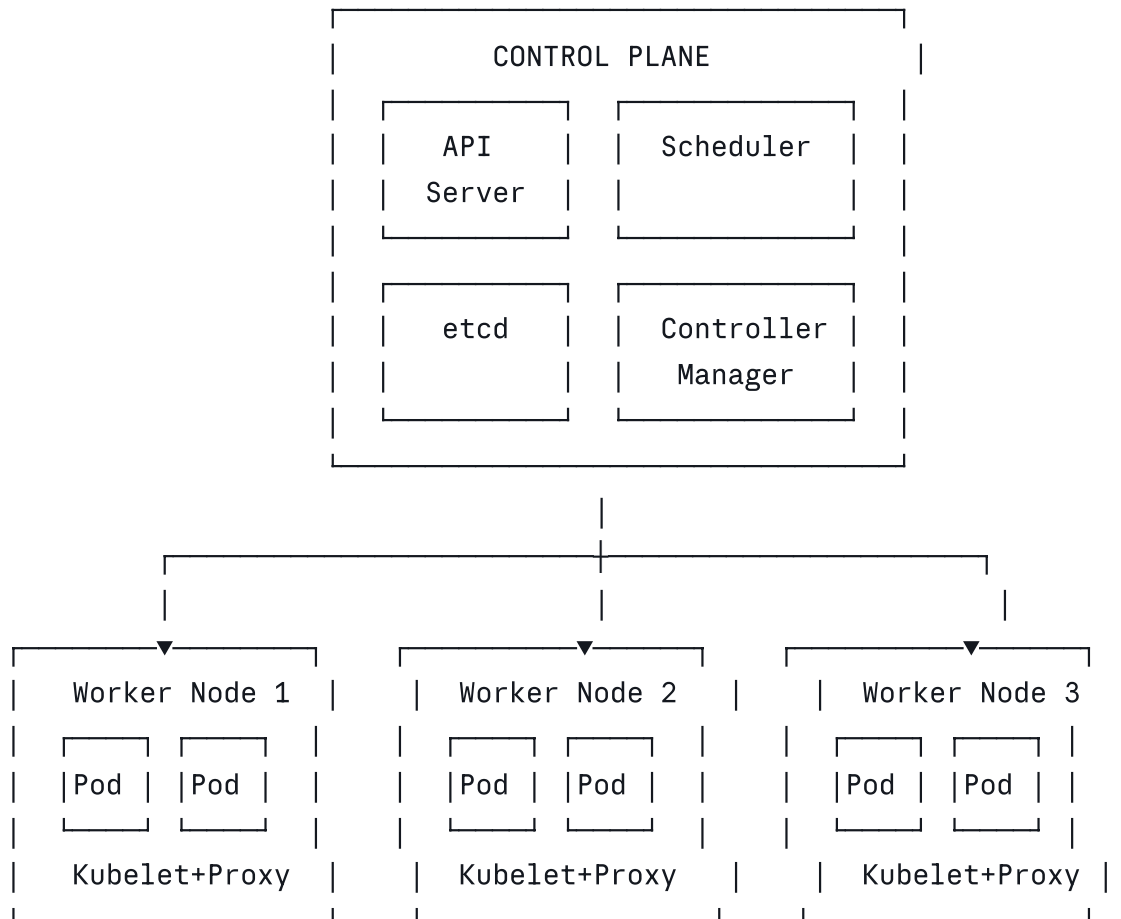
Kubernetes is an **open-source container orchestration platform** that automates deployment, scaling, and management of containerized applications.

 **Docker** = Runs individual containers  **Kubernetes** = Manages thousands of containers across multiple machines

Why Kubernetes for Java Microservices?

- **Auto-scaling** — Scale Spring Boot services based on traffic
- **Self-healing** — Restarts crashed containers automatically
- **Load balancing** — Distributes traffic across multiple pods
- **Zero-downtime deployments** — Rolling updates for your Java apps
- **Service discovery** — Microservices find each other automatically

Core Concepts (Architecture)



Key K8s Objects — Java Analogy

K8s Object	Description	Java Analogy
Pod	Smallest unit, runs 1+ containers	Single running JVM instance
Deployment	Manages replica pods + rolling updates	Service manager for your app
Service	Stable network endpoint to reach pods	Load balancer / API Gateway
ConfigMap	Stores non-sensitive config	<code>application.properties</code>
Secret	Stores sensitive config (base64)	Encrypted <code>application.properties</code>
Namespace	Logical cluster isolation	Java packages
Ingress	HTTP routing rules into cluster	Nginx reverse proxy
PersistentVolume	External storage for pods	Database storage
HPA	Auto-scales pods based on CPU/memory	Dynamic thread pool scaling

Essential kubectl Commands

Cluster & Context

bash

Check cluster info

kubectl cluster-info

View all contexts

kubectl config get-contexts

Switch context

kubectl config use-context my-cluster

Check current context

kubectl config current-context

View all nodes

kubectl get nodes

Node details

kubectl describe **node** <node-name>

Pod Commands

bash

List all pods (default namespace)

kubectl get pods

List pods in all namespaces

kubectl get pods -A

List pods in specific namespace

kubectl get pods -n production

Pod details with IP, node info

kubectl get pods -o wide

Describe pod (events, status)

kubectl describe pod <pod-name>

View pod logs

kubectl logs <pod-name>

Follow live logs

kubectl logs -f <pod-name>

Logs from specific container in pod

kubectl logs <pod-name> -c <container-name>

```
# Execute into pod
kubectl exec -it <pod-name> -- bash

# Delete pod (it will recreate if in Deployment)
kubectl delete pod <pod-name>
```

Deployment Commands

```
bash

# List deployments
kubectl get deployments

# Create deployment
kubectl create deployment my-app --image=my-java-app:1.0

# Apply YAML file
kubectl apply -f deployment.yaml

# Scale deployment
kubectl scale deployment my-app --replicas=5

# Update image (rolling update)
kubectl set image deployment/my-app my-app=my-java-app:2.0

# Check rollout status
kubectl rollout status deployment/my-app

# Rollout history
kubectl rollout history deployment/my-app

# Rollback to previous version
kubectl rollout undo deployment/my-app

# Rollback to specific revision
kubectl rollout undo deployment/my-app --to-revision=2

# Delete deployment
kubectl delete deployment my-app
```

Service Commands

```
bash

# List services
kubectl get services

# Expose deployment as service
```

```
kubectl expose deployment my-app --port=80--target-port=8080 --type=ClusterIP

# Describe service
kubectl describe service my-app

# Port forward (local testing)
kubectl port-forward svc/my-app 8080:80

# Delete service
kubectl delete service my-app
```

Namespace Commands

```
bash

# List namespaces
kubectl get namespaces

# Create namespace
kubectl create namespace production

# Set default namespace
kubectl config set-context --current --namespace=production

# Delete namespace (deletes all resources inside!)
kubectl delete namespace staging
```

ConfigMap & Secret

```
bash

# Create ConfigMap from literal
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=INFO

# Create ConfigMap from file
kubectl create configmap app-config --from-file=application.properties

# List ConfigMaps
kubectl get configmaps

# View ConfigMap content
kubectl describe configmap app-config

# Create Secret
kubectl create secret generic db-secret \
  --from-literal=DB_PASSWORD=mypassword \
  --from-literal=DB_USER=admin
```

```
# List Secrets
kubectl get secrets

# Decode secret value
kubectl get secret db-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 --decode
```

Monitoring & Debugging

```
bash

# Resource usage (needs metrics-server)
kubectl top pods
kubectl top nodes

# Get all resources in namespace
kubectl get all -n production

# Watch pods in real-time
kubectl get pods -w

# Get events (great for debugging)
kubectl get events --sort-by='.lastTimestamp'

# Describe any resource
kubectl describe <resource> <name>

# Delete resource from YAML
kubectl delete -f deployment.yaml

# Dry run (test without applying)
kubectl apply -f deployment.yaml --dry-run=client
```

YAML Manifests for Spring Boot App

1. Deployment

```
yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: spring-boot-app
  namespace: production
  labels:
    app: spring-boot-app
spec:
  replicas: 3 # 3 instances
```

```

selector:
  matchLabels:
    app: spring-boot-app
strategy:
  type: RollingUpdate          # Zero-downtime deploy
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
template:
  metadata:
    labels:
      app: spring-boot-app
spec:
  containers:
  - name: spring-boot-app
    image: my-java-app:1.0
    ports:
    - containerPort: 8080
    env:
    - name: SPRING_PROFILES_ACTIVE
      value: "production"
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:          # Inject from Secret
          name: db-secret
          key: DB_PASSWORD
  resources:
    requests:
      memory: "256Mi"
      cpu: "250m"
    limits:
      memory: "512Mi"
      cpu: "500m"
  livenessProbe:              # Restart if unhealthy
    httpGet:
      path: /actuator/health
      port: 8080
      initialDelaySeconds: 30
      periodSeconds: 10
  readinessProbe:             # Route traffic only when ready
    httpGet:
      path: /actuator/health/readiness
      port: 8080
      initialDelaySeconds: 20
      periodSeconds: 5

```

2. Service

yaml

```
apiVersion: v1
kind: Service
metadata:
  name: spring-boot-svc
  namespace: production
spec:
  selector:
    app: spring-boot-app          # Matches deployment pods
  ports:
    - protocol: TCP
      port: 80                    # Service port
      targetPort: 8080           # Container port
      type: ClusterIP            # Internal only
```

3. Ingress (External Access)

```
yaml

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: spring-boot-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: spring-boot-svc
                port:
                  number: 80
```

4. ConfigMap

```
yaml

apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  application.properties: |
    server.port=8080
```

```
logging.level.root=INFO
spring.profiles.active=production
```

5. HPA — Horizontal Pod Autoscaler

```
yaml

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: spring-boot-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: spring-boot-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70      # Scale when CPU > 70%
```

Top Interview Questions for 2 YOE Java Developer

Q1. What is the difference between Docker and Kubernetes?

Docker **runs** containers on a single machine. Kubernetes **orchestrates** containers across a cluster of machines — handling scaling, healing, networking, and updates automatically.

Q2. What is a Pod? Can it have multiple containers?

A Pod is the **smallest deployable unit** in K8s. It can have multiple containers that share the same network (same IP) and storage. Common pattern: Spring Boot app + log sidecar container in one pod.

Q3. Difference between ClusterIP, NodePort, and LoadBalancer?

Type	Access	Use Case
ClusterIP	Inside cluster only	Microservice-to-microservice
NodePort	Via Node IP + port	Dev/testing access
LoadBalancer	Public IP via cloud	Production external access

Q4. What is the difference between Liveness and Readiness Probe?

Liveness Probe — Is the container alive? If it fails, K8s **restarts** the container. Map to `/actuator/health`. **Readiness Probe** — Is the container ready to receive traffic? If it fails, K8s **removes it from load balancer** but doesn't restart. Critical for Spring Boot startup time.

Q5. How does a Rolling Update work in Kubernetes?

K8s gradually replaces old pods with new ones. With `maxSurge: 1` and `maxUnavailable: 0`, it brings up one new pod before taking down one old pod — ensuring **zero downtime**. If new pods fail, it pauses the rollout.

Q6. What is the difference between ConfigMap and Secret?

Both inject config into pods. **ConfigMap** stores plain text (env, properties). **Secret** stores base64-encoded sensitive data (passwords, tokens). Secrets can be encrypted at rest in production clusters.

Q7. How do you do zero-downtime deployment in Kubernetes?

- Use **RollingUpdate** strategy
- Set proper **Readiness Probes** (so traffic only goes to ready pods)
- Set **minReadySeconds** to stabilize
- Use `PodDisruptionBudget` to guarantee minimum availability

Q8. What is a Namespace and why use it?

Namespaces provide **logical isolation** within a cluster. Common pattern: `dev`, `staging`, `production` namespaces on the same cluster — with separate resource quotas and access controls.

Q9. How does Kubernetes handle auto-scaling?

HPA (Horizontal Pod Autoscaler) — Scales pod count based on CPU/memory/custom metrics. **VPA (Vertical Pod Autoscaler)** — Adjusts CPU/memory requests per pod. **Cluster Autoscaler** — Adds/removes worker nodes based on pending pods.

Q10. What happens when a Pod crashes in Kubernetes?

The **kubelet** on the worker node detects the failure. The **ReplicaSet** controller sees fewer pods than desired and creates a new one. If `restartPolicy: Always` (default), the pod is restarted. K8s uses **exponential backoff** for repeated crashes (CrashLoopBackOff).

Q11. How do you pass secrets securely to a Spring Boot app in K8s?

yaml


```
# Option 1: Environment variable from Secret
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: DB_PASSWORD

# Option 2: Mount as file
volumeMounts:
- name: secret-vol
  mountPath: /etc/secrets
volumes:
- name: secret-vol
  secret:
    secretName: db-secret
```

Q12. What is the difference between StatefulSet and Deployment?

Deployment	StatefulSet
Pods are identical & interchangeable	Each pod has stable unique identity
Random pod names	Ordered names (pod-0, pod-1)
For stateless apps (Spring Boot REST)	For stateful apps (MySQL, Kafka, Redis)
Can scale up/down in any order	Ordered, graceful scaling

Quick kubectl Cheat Sheet

```
bash

# Most used commands
kubectl get pods -n <ns> -o wide      # Pod list with details
kubectl logs -f <pod> -n <ns>        # Live logs
kubectl exec -it <pod> -- bash        # Shell access
kubectl apply -f <file>.yaml          # Deploy/update
kubectl rollout undo deployment/app   # Rollback
kubectl scale deployment app --replicas=5 # Scale
kubectl port-forward svc/app 8080:80 # Local testing
kubectl get events --sort-by='.lastTimestamp' # Debug events
kubectl top pods                      # CPU/Memory usage
kubectl describe pod <pod>            # Full pod info
```

Docker → Kubernetes Flow for Java App

1. Write Spring Boot App
↓
2. Create Dockerfile (multi-stage)
↓
3. Build & Push image → Docker Hub / ECR / GCR
↓
4. Write K8s YAML (Deployment + Service + Ingress)
↓
5. `kubectl apply -f .`
↓
6. K8s schedules pods on worker nodes
↓
7. HPA auto-scales based on traffic
↓
8. Rolling updates on new image versions

This covers everything a 2-year Java developer needs to know about Kubernetes for interviews! Focus especially on **Probes**, **Rolling Updates**, **Services types**, and **HPA** — these are the most commonly asked practical topics.

